Return your solutions on Moodle before **Sunday 12.4. at 23:59**. Your solutions must include intermediate steps and explain your reasoning.

**Exercise 6.1.** Solve the initial value problem

$$y'(t) = t + y, \quad y(0) = 1,$$

at $t = 1$ with Euler's method and step size (a) $h = 1$ and (b) $h = 0.5$. Compute the relative error for each step size. Plot the solution of the initial value problem and the $(t_i, y_i)$-points obtained with each step size on the same coordinate system and connect the points corresponding to each step size. You can use any tool you wish to obtain the exact solution.

**Solution.** The exact solution is

$$y(t) = 2e^t - t - 1 \implies y(1) = 2(e-1) \approx 3.44$$

(a) $h = 1$

Start from: $t_0 = 0$, $y_0 = 1$

First step:

$$t_1 = t_0 + h = 0 + 1 = 1,$$

$$y_1 = y_0 + h \cdot f(t_0, y_0) = 1 + 1 \cdot f(0,1) = 1 + 1(0+1) = 2$$

Relative error:

$$\rho_{h=1} = \left| \frac{y(1) - y_1}{y(1)} \right| = \left| \frac{2e-4}{2e-2} \right| \approx 41.8\%$$

(b) $h = 0.5$

Start from: $t_0 = 0$, $y_0 = 1$

First step:

$$t_1 = t_0 + h = 0 + \frac{1}{2} = \frac{1}{2},$$

$$y_1 = y_0 + h \cdot f(t_0, y_0) = 1 + \frac{1}{2} f(0,1) = 1 + \frac{1}{2}(0+1) = \frac{3}{2}$$

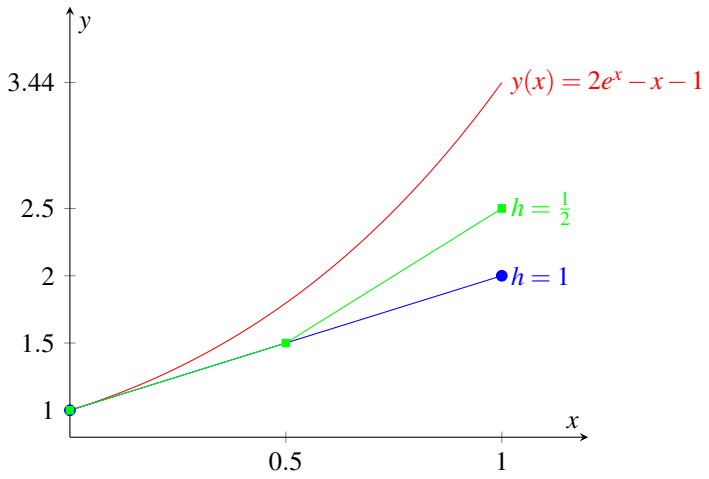
Second step:

$$t_2 = t_1 + h = \frac{1}{2} + \frac{1}{2} = 1,$$

$$y_2 = y_1 + h \cdot f(t_1, y_1) = \frac{3}{2} + \frac{1}{2} f(\tfrac{1}{2}, \tfrac{3}{2}) = \frac{3}{2} + \frac{1}{2}\left(\frac{1}{2} + \frac{3}{2}\right) = \frac{5}{2} = 2.5$$

Relative error:

$$\rho_{h=\frac{1}{2}} = \left| \frac{y(1) - y_2}{y(1)} \right| = \left| \frac{2e - 4.5}{2e-2} \right| \approx 27.3\%$$




□

**Exercise 6.2.** Use a programming language of your choice to write a program that solves an arbitrary initial value problem of the form

$$y'(t) = f(t, y(t)), \quad y(0) = y_0,$$

with Euler's method and an arbitrary step size. Implement the program so that you change the number of steps, $n$, rather than the step size, $h$. Apply the program to the initial value problem from Exercise 6.1 and determine how many steps are needed to obtain a relative error that is less than 10 %.

**Solution.**

```
from math import e

import numpy as np


def euler(odefun, y0, t0, tn, n):
    # initialisation
    ysol = np.empty(n + 1)
    ysol[0] = y0

    h = (tn - t0) / n # fixed step size
    tspan = np.r_[t0 : tn + h : h]

    # recurrence formula
    _euler = lambda t, y: y + h * odefun(t, y)

    # iterate
    for i in range(n):
        ysol[i + 1] = _euler(tspan[i], ysol[i])

    return tspan, ysol
```

```
# Problem 6.1
f = lambda t, y: t + y
t0 = 0
y0 = 1
tn = 1
y1 = 2 * (e - 1)


# Problem 6.2
tol = 0.1 # target relative error

# Initialisation
rho = float("Inf") # relative error
n = 0

# linearly probe the number of steps until rho < 10%
while rho >= tol:
    n += 1 # increase number of steps
    t, y = euler(f, y0, t0, tn, n) # solve
    rho = abs((y1 - y[n]) / y1)

# Result
print(f"With {n=} steps, the relative error is {rho:.3%}.")
```

With $n = 7$ steps, the relative error is $9.997\%$. □

**Exercise 6.3.** Write an implementation for the classical Runge–Kutta method that is analogous to the implementation of Euler's method in Exercise 6.2. Solve the initial value problem defined by

$$f(t,y) = \sin(ty) + \cos(t^2 + y^3)^4, \quad t_0 = 0, \quad y_0 = 1,$$

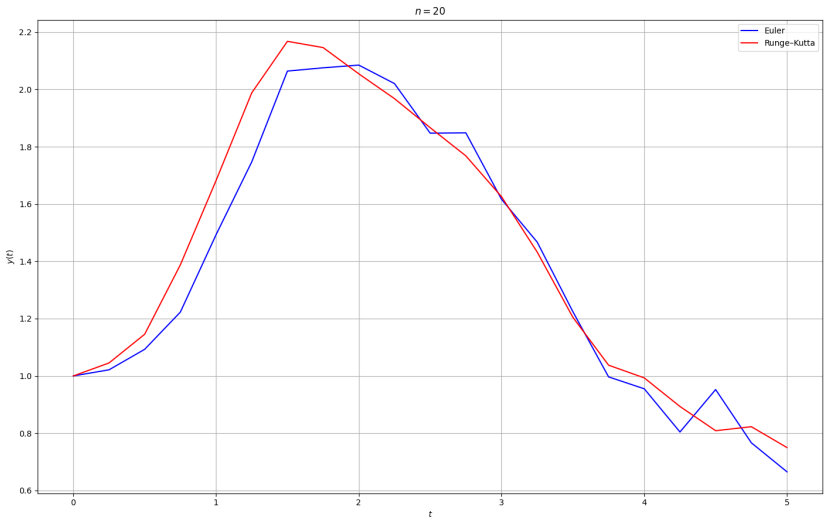
on the interval $[0,5]$ with both Euler's method and the classical Runge–Kutta method using $n = 20$, $n = 50$ and $n = 100$ steps. For each $n$, plot and compare the solutions you obtained with the two methods.

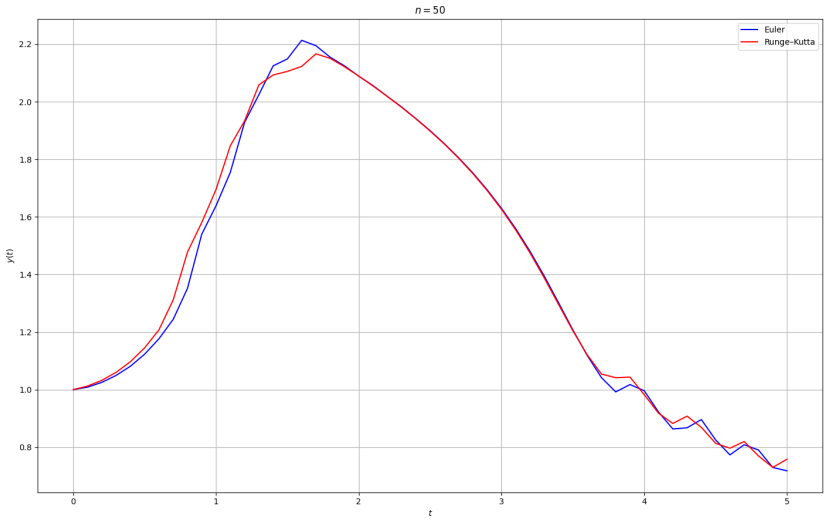
**Solution.**

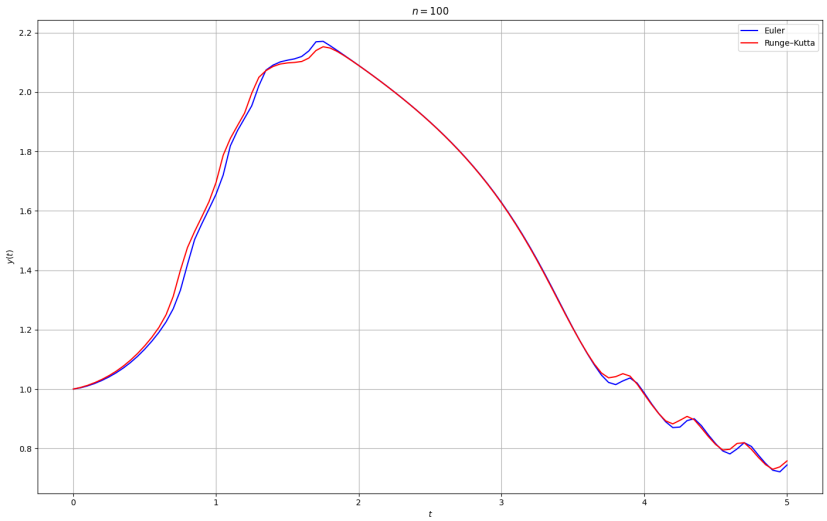
```
from functools import wraps

import numpy as np
import matplotlib.pyplot as plt


def odesolver(recurrenceform):

    @wraps(recurrenceform)
    def wrapper(odefun, y0, t0, tn, n):
        ysol = np.empty(n + 1)
        ysol[0] = y0
        h = (tn - t0) / n
        tspan = np.r_[t0 : tn + h : h]
```

```
        for i in range(n):
            ysol[i + 1] = recurrenceform(odefun, tspan[i], ysol[i], h)
        return tspan, ysol

    return wrapper


@odesolver
def euler(odefun, t, y, h):
    return y + h * odefun(t, y)


@odesolver
def rk4(odefun, t, y, h):
    k1 = odefun(t, y)
    k2 = odefun(t + h / 2, y + h / 2 * k1)
    k3 = odefun(t + h / 2, y + h / 2 * k2)
    k4 = odefun(t + h, y + h * k3)
    return y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4)


# Problem
f = lambda t, y: np.sin(t * y) + np.cos(t**2 + y**3) ** 4
t0 = 0
y0 = 1
tn = 5

# Constants
N = np.r_[20, 50, 100]

# Solve
for n in N:
    plt.figure()

    # Euler's method
    t, y = euler(f, y0, t0, tn, n)
    plt.plot(t, y, 'b', label="Euler")

    # classical Runge-Kutta
    t, y = rk4(f, y0, t0, tn, n)
    plt.plot(t, y, 'r', label="Runge-Kutta")

    # Plot
    plt.legend()
    plt.title(rf"$n = {n}$")
    plt.ylabel(r"$y(t)$")
    plt.xlabel(r"$t$")
    plt.grid(True)
    plt.show()
```

From the plots we can observe that the classical Runge–Kutta method converges faster than Euler's method to the true solution.


$n = 20$
Euler
Runge–Kutta
$y(t)$
$t$


$n = 50$
Euler
Runge–Kutta
$y(t)$
$t$




□

**Exercise 6.4.** The wolf and hare populations of a certain forest are determined by the pair of differential equations

$$x' = 0.2x - 0.01xy,$$
$$y' = -0.2y + 0.001xy.$$

(a) Determine the population sizes at time instance $t = 2$ using the Euler's method with step size $h = 1$ and initial conditions $x(0) = 100$ and $y(0) = 10$.

(b) Use a programming language of your choice to solve the initial value problem from part (a) with Euler's method and plot the population sizes on the time interval $[0, 100]$. Label the axes. Which of the variables $x$ and $y$ represents wolves and which hares?

**Solution.** Denote

$$f(t,x,y) = 0.2x - 0.01xy,$$

and

$$g(t,x,y) = -0.2y + 0.001xy.$$

Start from: $t_0 = 0$, $x_0 = 100$, $y_0 = 10$.
First step:

$$t_1 = t_0 + h = 0 + 1 = 1,$$

$$x_1 = x_0 + h \cdot f(t_0, x_0, y_0) = 100 + 1 \cdot f(0, 100, 10) = 100 + 0.2 \cdot 100 - 0.01 \cdot 100 \cdot 10 = 110,$$

$$y_1 = y_0 + h \cdot g(t_0, x_0, y_0) = 10 + 1 \cdot g(0, 100, 10) = 10 - 0.2 \cdot 10 + 0.001 \cdot 100 \cdot 10 = 9.$$

Second step:

$$t_2 = t_1 + h = 1 + 1 = 2,$$

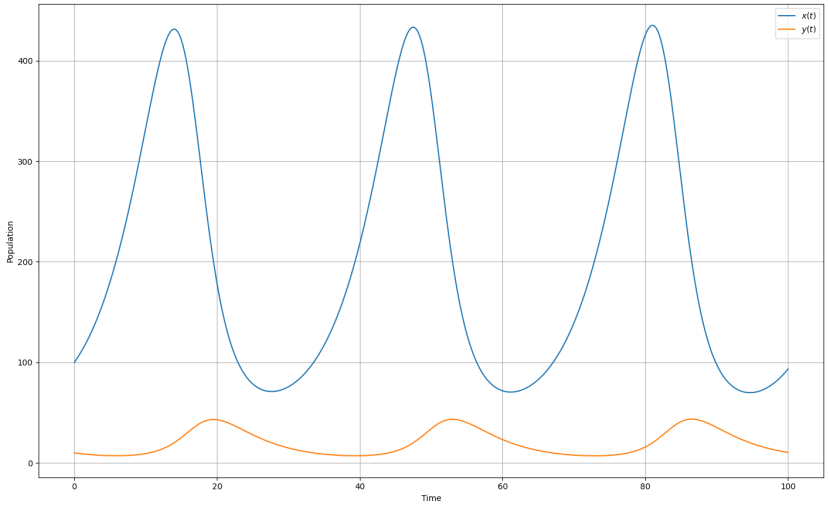
$$x_2 = x_1 + h \cdot f(t_1, x_1, y_1) = 110 + 1 \cdot f(0, 110, 9) = 110 + 0.2 \cdot 110 - 0.01 \cdot 110 \cdot 9 = 122.1$$
$$y_2 = y_1 + h \cdot g(t_1, x_1, y_1) = 9 + 1 \cdot g(0, 110, 9) = 9 - 0.2 \cdot 9 + 0.001 \cdot 110 \cdot 9 = 8.19.$$

The populations at time instance $t = 2$ are approximately:

$$x(2) \approx 122, \qquad y(2) \approx 8.$$

```
import numpy as np
import matplotlib.pyplot as plt


def euler(odefun, tspan, y0):
    # casting
    y0 = np.r_[y0]

    # initialisation
    n = len(tspan) - 1
    ysol = np.empty((n + 1, len(y0)))
    ysol[0, :] = y0

    # recurrence formula
    _euler = lambda t, y, h: y + h * odefun(t, y)

    # iterate
    for i in range(n):
        h = tspan[i + 1] - tspan[i]
        ysol[i + 1, :] = _euler(tspan[i], ysol[i, :], h)

    return ysol


# Parameters
f = lambda t, y: np.r_[
    0.2 * y[0] - 0.01 * y[0] * y[1], # x'(t)
    -0.2 * y[1] + 0.001 * y[0] * y[1], # y'(t)
]
h = 0.01 # step size
t = np.r_[0 : 100 + h : h]
y0 = (100, 10)

# Solve
y = euler(f, t, y0)

# Plot
plt.plot(t, y)
plt.legend((r"$x(t)$", r"$y(t)$"))
plt.ylabel("Population")
plt.xlabel("Time")
plt.grid(True)
plt.show()
```

By observing the plot, it's easy to tell that $x$ represents the population size of the preys (hares) and $y$ the the population size of the predators (wolves).




□

**Exercise 6.5.** Use Taylor's theorem to show that the *local truncation errors* of (a) Euler's method and (b) the midpoint method are of second order under appropriate assumptions on the function $y$. That is, prove that

$$y(t_1) - y_1 = O(h^2) \quad \text{as} \quad h \to 0,$$

where $t_1 = t_0 + h$.

**Solution.** We are interested to prove that

$$y(t_1) - y_1 = O(h^2) \quad \text{as} \quad h \to 0,$$

for Euler's method and the midpoint method. *We note that the term $y_1$ depends on the method's particular recurrence formula, while the term $y(t_1)$ does not. So let's start by checking what the term $y(t_1)$ looks like.* Let $y \in C^2(\mathbb{R})$ – *this is our sole assumption*. By the Taylor's theorem, $y(t)$ at the expansion point $t = t_0$ is

$$y(t) = P_1(t) + R_2(t) = y(t_0) + y'(t_0)(t - t_0) + \frac{y''(c_t)}{2}(t - t_0)^2,$$

for some $c_t$ between $t_0$ and $t$. Moreover, compute

$$y(t_1) = y(t_0) + y'(t_0)(t_1 - t_0) + \frac{y''(c_{t_1})}{2}(t_1 - t_0)^2$$

By definition $t_1 = t_0 + h \iff t_1 - t_0 = h$, therefore

$$y(t_1) = y(t_0) + y'(t_0)h + \frac{y''(c_{t_1})}{2}h^2.$$

(a) Euler's method recurrence formula is

$$y_{i+1} = y_i + h \cdot f(t_i, y_i) \qquad \text{where} \qquad f(t, y(t)) := y'(t) \quad \text{and} \quad h := t_1 - t_0.$$

Therefore, for an arbitrary initial value problem $y(t_0) = y_0$, the first iterate is

$$y_1 = y_0 + h \cdot f(t_0, y_0) = y(t_0) + h \cdot f(t_0, y(t_0)) = y(t_0) + h \cdot y'(t_0).$$

Thus the local truncation error is

$$y(t_1) - y_1 = \left[ y(t_0) + y'(t_0)h + \frac{y''(c_{t_1})}{2} h^2 \right] - \left[ y(t_0) + h \cdot y'(t_0) \right] = \frac{y''(c_{t_1})}{2} h^2.$$

Now claim that:

$$y(t_1) - y_1 = O(h^2) \quad \text{as} \quad h \to 0,$$

*Proof.* By the definition of Big-$O$ notation for functions, if $y(t_1) - y_1 = O(h^2)$ as $h \to 0$, then, for some $C \geq 0$ and $\delta > 0$, it must hold that $\left| y(t_1) - y_1 \right| \leq C \cdot h^2$ for all $h \in (-\delta, \delta)$. Observe that

$$\left| y(t_1) - y_1 \right| = \left| \frac{y''(c_{t_1})}{2} h^2 \right| = \left| \frac{y''(c_{t_1})}{2} \right| \cdot \left| h^2 \right| = \left| \frac{y''(c_{t_1})}{2} \right| h^2,$$

therefore set $C = \left| \frac{y''(c_{t_1})}{2} \right|$. This shows that $y(t_1) - y_1 = O(h^2)$ for all $h \in \mathbb{R}$.

(b) The midpoint method recurrence formula is

$$y_{i+1} = y_i + h \cdot f\left( t_i + \tfrac{h}{2}, y_i + \tfrac{h}{2} f(t_i, y_i) \right) \quad \text{where} \quad f(t, y(t)) := y'(t) \quad \text{and} \quad h := t_1 - t_0.$$

Therefore, for an arbitrary initial value problem $y(t_0) = y_0$, the first iterate is

$$y_1 = y_0 + h \cdot f\left( t_0 + \tfrac{h}{2}, y_0 + \tfrac{h}{2} f(t_0, y_0) \right) = y(t_0) + h \cdot f\left( t_0 + \tfrac{h}{2}, y(t_0) + \tfrac{h}{2} y'(t_0) \right).$$

Note that $y(t_0) + \frac{h}{2} y'(t_0)$ is essentially the first-degree Taylor polynomial $P_1(t_0 + h/2)$ of $y(t)$ expanded at $t = t_0$. Therefore, by the Taylor's theorem, we have

$$y(t_0) + \frac{h}{2} y'(t_0) = y\left( t_0 + \tfrac{h}{2} \right) - \frac{y''(\varepsilon)}{8} h^2$$

for some $\varepsilon$ between $t_0$ and $t_0 + h/2$. Substituting it into $y_1$ yields

$$y_1 = y(t_0) + h \cdot f\left( t_0 + \tfrac{h}{2}, y\left( t_0 + \tfrac{h}{2} \right) - \frac{y''(\varepsilon)}{8} h^2 \right).$$

*Unfortunately we cannot further simplify by substituting $f(\dots)$ with $y'(t_0 + h/2)$ as before due to the presence of the remainder term. But let's take a closer look at $f(t, y)$.*

For a fixed $t = \tilde{t}$, $f$ can be read as a function of one variable, i.e. $g(y) = f(\tilde{t}, y)$. Since $y$ is twice continuously-differentiable and $f := y'$, then $f'$ exists and therefore also $g'$. By the Taylor's theorem, $g(y)$ at expansion point $y = \tilde{y}$ is

$$g(y) = g(\tilde{y}) + g'(c_y)(y - \tilde{y})$$

for some $c_y$ between $\tilde{y}$ and $y$. If $y = \tilde{y} + \delta$ for some $\delta \in \mathbb{R}$, then

$$g(\tilde{y} + \delta) = g(\tilde{y}) + g'(c_y)\delta$$

and therefore

$$f(\tilde{t},\tilde{y}+\delta) = f(\tilde{t},\tilde{y}) + g'(c_y)\delta.$$

Let $\tilde{t} = t_0 + h/2$, $\tilde{y} = y(t_0 + h/2)$, and $\delta = -y''(\varepsilon)h^2/8$. Then we can conclude that

$$\begin{aligned} f\left(\left[t_0+\tfrac{h}{2}\right], \left[y\left(t_0+\tfrac{h}{2}\right)\right] + \left[-\frac{y''(\varepsilon)}{8}h^2\right]\right) &= f\left(t_0+\tfrac{h}{2}, y\left(t_0+\tfrac{h}{2}\right)\right) - g'(c_y)\frac{y''(\varepsilon)}{8}h^2 \\ &= y'\left(t_0+\tfrac{h}{2}\right) - g'(c_y)\frac{y''(\varepsilon)}{8}h^2. \end{aligned}$$

Substituting it into $y_1$ yields

$$y_1 = y(t_0) + h\cdot\left[y'\left(t_0+\tfrac{h}{2}\right) - g'(c_y)\frac{y''(\varepsilon)}{8}h^2\right] = y(t_0) + h\cdot y'\left(t_0+\tfrac{h}{2}\right) - g'(c_y)\frac{y''(\varepsilon)}{8}h^3.$$

Finally, by the Taylor's theorem, $y'(t)$ about the expansion point $t = t_0$ is

$$y'(t) = y'(t_0) + y''(\alpha)(t - t_0)$$

for some $\alpha$ between $t_0$ and $t$. Therefore $y'(t_0 + h/2)$ is

$$y'\left(t_0+\tfrac{h}{2}\right) = y'(t_0) + y''(\alpha)\frac{h}{2}.$$

Substituting it into $y_1$ yields

$$\begin{aligned} y_1 &= y(t_0) + h\cdot\left[y'(t_0) + y''(\alpha)\frac{h}{2}\right] - g'(c_y)\frac{y''(\varepsilon)}{8}h^3 \\ &= y(t_0) + h\cdot y'(t_0) + y''(\alpha)\frac{h^2}{2} - g'(c_y)\frac{y''(\varepsilon)}{8}h^3. \end{aligned}$$

Thus the local truncation error is

$$\begin{aligned} y(t_1) - y_1 &= \left[y(t_0) + y'(t_0)h + \frac{y''(c_{t_1})}{2}h^2\right] - \left[y(t_0) + h\cdot y'(t_0) + y''(\alpha)\frac{h^2}{2} - g'(c_y)\frac{y''(\varepsilon)}{8}h^3\right] \\ &= \frac{y''(c_{t_1}) - y''(\alpha)}{2}h^2 + g'(c_y)\frac{y''(\varepsilon)}{8}h^3. \end{aligned}$$

By Theorem 2.18 we conclude that

$$y(t_1) - y_1 = O(|h|^2) \quad \text{as} \quad h \to 0,$$

i.e.

$$y(t_1) - y_1 = O(h^2) \quad \text{as} \quad h \to 0$$

as requested.

□